

Forensic Analysis Report

Project - Memory Forensics and Volatile Artefact Analysis

Project Team #09

Team Member	Student ID
Aashish Anand	645876
Joshua Crisford	574082
Emily Tinsley	652275
Poorvika Kamberkavi	641799

Table of Contents

Table of Contents	2
1. Introduction	5
2. What Is Memory Analysis?	7
2.1 When and Why Memory Acquisition Is Used	7
2.2 Random Access Memory (RAM) Analysis in Forensic Investigations	7
3. Procedures	9
3.1 Setup and Configuration of Ubuntu VM	9
3.2 Installation of Volatility 3	9
3.3 Setup and Configuration of Windows 10 VM	10
3.4 Capturing VM Memory Dump	12
3.5 Malware Selection, Handling, and Execution	12
3.6 Creating SHA-256 Hashes	13
4. Methodological Rationale	14
4.1 VM Resource Choices	14
4.2 Network & Sharing Controls	14
4.3 Volatility Installation	14
4.4 Memory Capture Method and Conversion	14
4.5 Malware Selection	14
5. Python Automation script	16
5.1 Overview	16
5.2 Automation Workflow	16
5.3 Integration and Error Handling	16
5.4 Python Involvement	17
5.5 Outcome	17
5.6 Memory Comparison Script (Stretch Objective 7)	17
5.6.1 Purpose	17
5.6.2 How It Works (Step-by-Step)	18
5.6.3 Forensic Significance	18
Note	18

6. Results and Analysis	19
6.1 Clean Memory Image.....	19
6.1.1 Overview and Image Integrity.....	19
6.1.2 System Information and Environment Artefacts	19
6.1.3 Process and Execution Artefacts	19
6.1.4 Network and Communication Artefacts	19
6.1.5 Credential and Security Artefacts.....	20
6.1.6 File System and Persistence Artefacts.....	20
6.1.7 Baseline Artefact Reference Table.....	20
6.1.8 Interpretation Summary.....	21
6.2 Infected Memory Image	21
6.2.1 Overview	21
6.2.2 Consistency of Failures.....	21
6.2.3 Nature and Extent of Failures.....	21
6.2.4 Dataset Overview and Verification	22
6.2.5 Interpretation Summary.....	22
6.3 Analysis of Failure Causes.....	22
6.3.1 Overview	22
6.3.2 Volatility Dependencies and Symbol Table Requirements	23
6.3.3 Kernel Disruption and Address Translation Failures	23
6.3.4 Tool Limitations and Reasoning for Error Propagation.....	23
6.3.5 Summary.....	24
6.4 Forensic Implications of PlugX Behaviour	24
6.4.1 Technical Functionality.....	24
6.4.2 Impact on Volatility and Forensic Results	25
7. Discussion.....	26
7.1 Comparison to Literature and Case Studies	26
7.2 Recommendations and Future Improvements	27
7.3 Reflective Analysis.....	28
8. Conclusion	29
9. References.....	30

10. Appendix	32
Appendix A - Hash Verification Logs.....	32
Appendix B – Software and Forensic Tools Used.....	32

1. Introduction

This report presents the process, justifications, and outcomes of the Memory Forensics and Volatile Artefact Analysis project created as part of KIT325 Advanced Cybersecurity and eForensics. Memory forensics involves producing and identifying the contents of Random-Access Memory (RAM) to ideally reproduce and analyse the state of a system now when a memory image was captured (NIST 2023). The project focused on extracting and analysing volatile memory to identify forensic artefacts created by malicious software within a controlled environment.

The intention of the project was to simulate a real-life malware investigation using current forensics techniques and open-source tools. Due to working with dangerous malware, a secure virtual environment was set up using Oracle's VirtualBox software, containing an Ubuntu virtual machine for analysis, and a Windows 10 victim machine to analyse. The Volatility 3 software was chosen to parse the captured memory images and analyse extracted artefacts relevant to system activity and malware actions (Volatility Foundation, 2024). The PlugX Remote Access Trojan (RAT) was used within the Windows 10 VM to generate data for investigation.

The report documents the complete process of the project, including setup, malware handling, memory capture, and artefact discovery and analysis. It explains the methodologies applied, presents and discusses the discovered artefacts from analysis, and confers their forensic significance. An automation script is included to demonstrate how these techniques could be replicated efficiently in other forensic investigations. Finally, the report examines the challenges and limitations encountered during the project, discusses potential improvements, and concludes with reflections on the value of memory forensics in investigations.

The scope of this project involves researching and selecting appropriate Volatility 3 plugins for analysing a captured memory image of a Windows 10 operating system, researching and selecting malware, utilising the selected Volatility plugins in comparing captured memory images before and after injecting chosen malware into the VM, and constructing a full report on the memory analysis conducted. The deliverables of this project include a memory imaging log (Sections 4 and Appendix A and B), an analysis output archive, a memory artefact cheat sheet, a forensic report, and a Python script used to automate memory analysis using Volatility 3. Many of these deliverables can be found in the git repository associated with this project.

Safety precautions were taken before injecting malware into the VM to ensure there was no file sharing between the VM and the host computer in order to prevent the spread of malware through compromised data. These included isolation of the VM file system and the disabling of Wi-Fi and other network connections between the VM and the host

computer. These precautions allow memory captures of a VM infected with malware to be collected without any negative impacts on the host computer system or other live systems.

2. What Is Memory Analysis?

2.1 When and Why Memory Acquisition Is Used

Unlike disk-based forensics, volatile memory can provide transient data, including running processes, network connections, encryption keys, and injected code that is never written onto persistent storage (Lign et al., 2014). This type of data disappears when the system is powered off, meaning the timing and methods of memory acquisition are important to an investigation.

Live memory acquisition takes place while the system is running, which allows investigators to harvest that crucial transient data before it is lost. This approach is common in incident-response scenarios, as maintaining access to volatile evidence often offsets the risk of a slightly altered system image. Alternatively, post-mortem memory acquisition is done after the system is shut down or isolated. This prioritises forensic accuracy and data integrity rather than ensuring the completeness of volatile data. The post-mortem method is used primarily when evidence preservation is a crucial objective (Case & Richard, 2017).

Choosing between live and post-mortem memory acquisition, therefore, depends on intended investigative outcomes: real-time evidence of malicious activity, or verifiable and repeatable examination. This project uses live memory acquisition for data capture to allow the team to observe as many in-memory artefacts, active processes, injected code, and network connections that would have been lost after system shutdown as possible. Due to the experiment being performed within a controlled and isolated environment, the minor risks involved are vastly outweighed by the possibility of producing realistic artefacts for analysis (NIST, 2023).

2.2 Random Access Memory (RAM) Analysis in Forensic Investigations

Memory forensics has been proven to be a useful technique in many investigations where disk analysis has not been able to discover malicious activity. The PlugX Remote Access Trojan, the same malware used in this project, is a malware family used in targeted attacks against government and corporate networks in the past. Researchers analysing PlugX infections in past investigations have discovered that the malware performs DLL sideloading and code-injecting exploits into legitimate Windows processes, making it essentially invisible to file-based analysis (Cybereason, 2022). Using volatile-memory examination methods, investigators identified injected modules, hidden threads, and command-and-control (C2) communication through analysis of running processes and network artefacts stored in RAM (Volatility Labs, 2015). These

investigations demonstrate the ability of memory forensics to reveal behaviour that does not exist within static storage.

During a similar famous malware outbreak, the WannaCry ransomware, investigators used memory acquisition which allowed them to discover the encryption processes and recover encryption keys that would have been lost after shutdown (CERT-EU, 2018). This is another demonstration of RAM analysis providing crucial insight into system state, providing investigators the ability to reconstruct attacker activity and analyse infection mechanisms.

3. Procedures

3.1 Setup and Configuration of Ubuntu VM

1. Create folder located on the Desktop called “VM and eForensic work”.
2. Download Ubuntu 25.04 for Intel from <https://ubuntu.com/download/desktop> and move download to the newly created folder.
3. Download VirtualBox 7.2.2 installer for Windows hosts from <https://www.virtualbox.org/wiki/Downloads> and install VirtualBox 7.2.2
4. Open VirtualBox when installation is complete and select the “New” icon in the top right corner. Type “Ubuntu” into the name field and enter “C:\Users\[username]\Desktop\VMs and Cybersecurity\ubuntu-25.04-desktop-amd64.iso”, replacing [username] with username of the host machine. Check the box to “Proceed with Unattended Installation” and click next.
5. Enter a Username and Password that will be used as account credentials to login into the Ubuntu VM. Credentials used for this instance “ubuntu_user” and “HelloUbuntu” for the username and password, respectively. Check the box to “Guest Additions” and click next.
6. Set Base Memory to 5000MB, number of CPUs to 3, and click next.
7. Set Disk Size to 25.00GB and click next. Ensure all information is correct on the summary page and click finish.
8. Open Ubuntu VM by selecting the Ubuntu VM in the list to the left and selecting the start icon.
9. Wait for the VM to install the Ubuntu memory image and log in using the previously set credentials.
10. Inside the Ubuntu VM, open the terminal and enter the following commands to ensure system is running on most updates version.

```
sudo apt update && sudo apt upgrade -y
sudo snap refresh
```
11. Close the Ubuntu VM by clicking the cross in the top right corner, select the “Power off the machine” option, and click ok.
12. In VirtualBox click the settings icon, click Network in the left menu, and uncheck the “Enable Network Adapter” box.
13. Click Shared Folders in the left menu and ensure that there are no shared folders. Click ok.

3.2 Installation of Volatility 3

1. Enable Network connection to the Ubuntu VM by selecting Ubuntu in the left menu of VirtualBox clicking the settings icon, click Network in the left menu, and check the “Enable Network Adapter” box.

2. Open Ubuntu VM by selecting Ubuntu in the left menu of VirtualBox and clicking the start icon.
3. Log into the Ubuntu VM with the previously established credentials and open the terminal within Ubuntu.
4. Enter the following commands to ensure the required python files are installed.

```
sudo apt install python3 git -y
```
5. Enter the following command to install Volatility 3 (following command is to be entered as a single line).

```
git clone  
https://github.com/volatilityfoundation/volatility3.git
```
6. Enter the following command navigate to the Volatility 3 directory.

```
cd volatility3
```
7. Enter the following command to install all requirements for volatility 3.

```
pip3 install -r requirements.txt
```
8. Enter following command the ensure installation.

```
python3 vol.py -h
```
9. Close the Ubuntu VM by clicking the cross in the top right corner, select the “Power off the machine” option, and click ok.
10. Disable Network connection to the Ubuntu VM again by selecting Ubuntu in the left menu of VirtualBox clicking the settings icon, click Network in the left menu, and uncheck the “Enable Network Adapter” box.

3.3 Setup and Configuration of Windows 10 VM

1. Download Windows 10 media creation tool from <https://www.microsoft.com/en-us/software-download/windows10> and open the tool.
2. Accept license terms.
3. Select the “Create installation media (USB flash drive, DVD or ISO file) for another PC” option and click next.
4. Click next again.
5. Select the “ISO file” option and click next. Select the VM and eForensic work folder as the destination file for the ISO file when prompted by a files window and click open.
6. Click finish.
7. Open VirtualBox and select the “New” icon in the top right corner. Type “Windows 10” into the name field and enter
“C:\Users\[username]\Desktop\VMs and Cybersecurity\Window.iso”, replacing [username] with username of the host machine. Check the box to “Proceed with Unattended Installation” and click next.

8. Expand “Set up unattended guest OS installation” header and enter a Username and Password that will be used as account credentials to login into the Windows 10 VM. Credentials used for this instance “vboxuser” and “HelloWindow10” for the username and password, respectively.
9. Copy the default product key for Windows 10 Home from <https://gist.github.com/rvrsh3ll/0810c6ed60e44cf7932e4fbae25880df#default-product-keys> and past into the product key field. Click finish.
10. Open Windows 10 VM by selecting the Windows 10 VM in the list to the left and selecting the start icon.
11. Disable Real-time Protection by opening Windows Security, and opening Virus & threat protection tab. Close Windows Security.
12. Open the command prompt as administrator, and enter the following commands to install Local Group Policy Editor (this step is not required for windows pro or enterprise). Close the command prompt.

```
FOR %F IN ("%SystemRoot%\servicing\Packages\Microsoft-Windows-GroupPolicy-ClientTools-Package~*.mum") DO (DISM /Online /NoRestart /Add-Package:"%F")
FOR %F IN ("%SystemRoot%\servicing\Packages\Microsoft-Windows-GroupPolicy-ClientExtensions-Package~*.mum") DO (DISM /Online /NoRestart /Add-Package:"%F")
```

13. Press window + R, enter “gpedit.msc”, and press okay.
14. Disable Windows Defender by navigating to “Computer Configuration > Administrative Templates > Windows Components > Microsoft Defender Antivirus” in group policy editor, double-clicking "Turn off Microsoft Defender Antivirus" policy and set it to "Enabled".
15. Navigate into “Computer Configuration > Administrative Templates > Windows Components > Microsoft Defender Antivirus>Real-time Protection” in group policy editor, double-clicking "Monitor file and program activity on your computer" policy and set it to "Disabled".
16. Navigate into “Computer Configuration > Administrative Templates > Windows Components > Windows Update” in group policy editor, double-clicking "Configure Automatic Updates" policy and set it to "Disabled".
17. Open File Explorer and click file, click “Change folder and search options”, click the view tab, disable “Hide extensions for known file types” and check “Show hidden files, folders, and drives”. Close File Explorer.
18. Disable Address Space Layout Randomization (ASLR) by opening Registry and navigate to “HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Memory Management” and create a “REG_DWORD” type key named “MoveImages”. Close Registry

19. Open Control Panel, click System and Security, click Windows Defender Firewall, click "Turn Windows Defender Firewall on or off". Close the Control Panel.
20. Take a snapshot of the VM by clicking Machine in the top menu and click "Take Snapshot...". Name the snapshot "Clean Snapshot".

3.4 Capturing VM Memory Dump

1. Open VirtualBox as an administrator.
2. Open Windows 10 VM by selecting the Windows 10 VM in the list to the left and selecting the start icon.
3. In the VM, download FTKImager installer from <https://www.exterro.com/ftk-product-downloads/ftk-imager-4-7-3-81>
4. Open the FTKImager installer and Install FTKImager onto the VM desktop.
5. Open FTKImager as an administrator.
6. Expand the file menu from the tool bar at the top and select "Capture Memory..."
7. Select the destination file to be the VM desktop and the file name to be "Cleanmemdump.mem".
8. Click "Capture Memory".
9. In VirtualBox, click the file manager icon in the Windows 10 VM information. Login using the Windows 10 credentials at the bottom of the guest file system panel.
10. Locate the memory dump file in the guest file system by navigating to /home/window_user/Desktop on the guest file system panel.
11. Click the copy from guest to host icon located between the two file system panels. Close file sharing session.
12. Repeat steps 5 to 11 after malware has been injected, replacing the captured memory filename to "PlugXInfectedmemdump.mem".

3.5 Malware Selection, Handling, and Execution

1. Open Windows 10 VM by selecting the Windows 10 VM in the list to the left and selecting the start icon.
2. Access the malware repository:
<https://github.com/ytisf/theZoo/tree/master/malware/Binaries>
3. Choose the PlugX malware sample in the Binaries folder.
4. Download the sample archive (zip) from the repository to a shared project folder accessible by the host and the rest of the team.
5. Extract the sample from the .zip file using the password "infected" when prompted.
6. Open the executable file.

7. Verify the malware is running by confirming the expected process is present in the VM (e.g., visible in Task Manager).

3.6 Creating SHA-256 Hashes

1. Create a hash for the memory image using the below command in the Ubuntu VM terminal (replacing [mem file] with path to memory capture to be hashed, and replacing [filename] with appropriate filename for file containing the hash output):

```
sha256sum [mem file] > ./Desktop/[filename]
```

4. Methodological Rationale

4.1 VM Resource Choices

These resource settings provide a stable environment that is large enough to run a Windows 10 OS and the malware while keeping host resource usage reasonable. Allocating 5 GB of RAM and a few vCPUs ensures Windows will not behave abnormally due to resource starvation, which could alter volatile artefacts and reduce the integrity and reliability of the memory captures (Oracle; REMnux n.d.).

4.2 Network & Sharing Controls

Disabling network access and host-guest sharing ensures the malware sample cannot make uncontrolled external connections or escape the isolated environment. This control is core to any malware experiment as it removes risk to the host and the wider network, ensuring that network or file activity is contained and reproducible within the isolated environment (REMnux; Zeltser n.d.).

4.3 Volatility Installation

Installing Volatility 3 on the dedicated Ubuntu VM keeps the analysis separate from the infected VM. Using standard Linux analysis environments simplifies management of things like Python and Git and reduces possible interference with Windows artefacts. It is also common in digital forensics workflows and helps keep the toolchain reproducible across different hosts (Volatility Foundation; REMnux n.d.).

4.4 Memory Capture Method and Conversion

Using VirtualBox's built-in `dumpvmcore` tool allows the entire virtual machine memory to be captured from the hypervisor perspective. This ensures the dump reflects the VM's live RAM state accurately without modifying the VM OS. Volatility 3 requires a raw memory image to analyse, so converting the core dump provides format compatibility and prevents analysis errors. Recording the filenames and hashes after conversion ensures integrity for later verification (Oracle; Volatility Foundation n.d.).

4.5 Malware Selection

The malware sample used in this project was selected from theZoo, a public malware repository hosted on GitHub that provides access to verified malware samples for educational and research purposes. Each sample in the Zoo is archived and documented for use in legitimate forensic and analytical work. This makes it a highly suitable and ethical source for obtaining malware for this project (ytisf n.d.).

From this repository, the PlugX malware sample was chosen. PlugX is a Remote Access Project (RAT) that has been very active in real-life cyberattacks for nearly ten years, and it is well known for its memory-resident behaviour. It uses DLL sideloading and code injection techniques to conceal its operations, which makes it particularly suitable for volatile artefact study within system memory. This choice ensures meaningful and realistic forensic data to discover and discuss within the project, as its activity leaves identifiable traces in RAM that can be captured and analysed using Volatility 3 (Cybereason 2022; Malpedia 2022).

5. Python Automation script

To streamline the forensic analysis process and reduce manual intervention, a set of automation scripts was developed using Bash and Python to run Volatility 3 modules systematically across memory images. These scripts collectively handle metadata extraction, process scanning, and artefact collection to produce consistent and reproducible forensic outputs.

5.1 Overview

The automation framework consists of three modular scripts:

1. `vol3_info.sh` – Performs baseline analysis of the memory image to identify system profile, kernel details, registry hives, and optional time zone data.
2. `run_vol3_scan.sh` – Executes core Volatility 3 plugins focusing on process listings, network connections, and DLL enumeration.
3. `vol3_artifacts.sh` – Focuses on user session reconstruction, credential recovery, malware detection, and timeline generation for incident response.

These scripts are executed sequentially or individually, depending on the analysis stage, with outputs stored in timestamped directories under the `outputs/` folder for clear traceability.

5.2 Automation Workflow

Each script automatically:

- Detects the memory image path and generates an output directory with a UTC timestamp.
- Executes selected Volatility 3 plugins relevant to its category.
- Logs plugin execution results and warnings into per-plugin text files.
- Generates summary reports (`README.txt` and `PROJECT_ANALYSIS_SUMMARY.txt`) to guide the investigator's review process.

This approach ensures repeatability, transparency, and efficient data handling when processing large or multiple memory captures.

5.3 Integration and Error Handling

The automation includes:

- **Structured logging:** Each plugin's output is saved separately for ease of analysis.
- **Error handling:** Failed plugin runs are flagged with warnings but do not interrupt execution.
- **Optional modules:** Some plugins (e.g., windows.mbr, registry.find_key) are treated as optional to maintain compatibility with varying Windows memory profiles.

5.4 Python Involvement

Although the orchestration is written in Bash, each script internally invokes the Python-based Volatility 3 framework via:

```
python3 ~/volatility3/vol.py -f <image> <plugin>
```

This enables the use of Python's extensible plugin architecture for memory forensics, while Bash ensures automation and workflow control.

5.5 Outcome

The automated setup provides:

- A consistent baseline analysis across all memory samples.
- Faster malware and artefact extraction for comparative analysis between clean and infected images.
- A structured dataset suitable for deeper manual investigation and inclusion in the project report.

All automation scripts developed for this project are version-controlled in our GitHub repository under the scripts/ directory. Each script includes a companion Markdown file (<script_name>.md) that documents the script's objective, required dependencies (Volatility 3 or local vol.py, Python packages), step-by-step usage examples, the list of Volatility plugins invoked, and the structure of the generated outputs/ folder. This documentation ensures reproducibility and lets other team members or assessors run the same automated analysis reliably; example invocation commands and troubleshooting tips are provided in each .md.

5.6 Memory Comparison Script (Stretch Objective 7)

5.6.1 Purpose

This script performs an automated forensic comparison between two Windows memory snapshots: a *clean baseline image* and a *post-infection image*.

It was designed for Stretch Objective 7 of the project: "*Compare snapshots before and after malware injection to detect new processes, network connections, or modules introduced by the malware.*"

5.6.2 How It Works (Step-by-Step)

1. Takes two input memory images:
`./vol3_compare.sh clean_image.raw infected_image.raw`
2. Runs a selected set of **Volatility 3 plugins** (pslist, netscan, modules, handles, cmdline) on both images.
3. Extracts and sorts relevant artefacts (process names, network connections, kernel modules).
4. Performs **line-by-line diff comparisons** between clean and infected outputs.
5. Generates a consolidated forensic report:
`MALWARE_COMPARISON_REPORT.txt` under
`outputs/comparison_<timestamp>/`
6. Summarises key findings under:
 - New processes possibly indicating injected executables
 - New network connections suggesting C2 or exfiltration
 - New kernel modules implying rootkit activity
 - Correlated forensic implications for incident response

5.6.3 Forensic Significance

This script operationalises differential memory forensics, a powerful approach for identifying malware artefacts when baseline snapshots are available.

Even though the infected memory image failed to validate due to suspected corruption, the script's methodology remains valid.

It represents a proof of concept for automated before/after memory analysis, showing how volatile artefacts could be systematically compared to isolate malware behaviour.

In a real-world investigation, this approach would allow an analyst to:

- Identify process injection or malicious persistence
- Detect unauthorised network activity
- Uncover newly loaded modules introduced by malware

Note

Due to issues validating the infected image's plugin requirements, the script was not executable for full comparison testing.

However, its inclusion in the project repository reflects the intended direction for automated malware artefact correlation, reinforcing the project's forensic and analytical objectives.

6. Results and Analysis

6.1 Clean Memory Image

6.1.1 Overview and Image Integrity

The clean Windows 10 memory image serves as a foundation for analysis, specifically in identifying and interpreting artefacts to be later compared with the PlugX-infected image. The image was verified using matching SHA-256 hashes and was ingested by all Volatility 3 plugins successfully. This shows the kernel structures were intact and interpretable, which is sufficient for a reliable reference point for expected artefact locations and plugin functionality. This foundation essentially demonstrates how a healthy system's memory should appear before a malware infection.

6.1.2 System Information and Environment Artefacts

System information plugins such as `windows.info`, `windows.verinfo`, `windows.hivelist` correctly identified the operating systems as Windows 10 (10.0.19041, x64) running in a VirtualBox environment. The registry hives `SYSTEM`, `SOFTWARE`, `SAM`, `SECURITY`, and `DEFAULT` were all located, as well as user hives for a default account, `windows10_user`. These show that registry hierarchy and kernel mapping are complete and uncorrupted, allowing plug-ins to read configuration, credential, and user activity artefacts. The result is a baseline for comparing registry and persistence-related artefacts in the infected image.

6.1.3 Process and Execution Artefacts

Process plugins, `pslist`, `psscan`, `dlllist`, and `cmdline` showed normal active process activity for a standard Windows session and included `System`, `smss.exe`, `winlogon.exe`, `svchost.exe`, `explorer.exe` and `FTK Imager.exe`. These, aside from FTK Imager used for capturing the memory image, represent normal system operations taking place at the time of image acquisition. Related threads and DLLs were correctly indicated, and no hidden processes were detected. This provides a suitable foundation for identifying new or anomalous processes that are spawned or injected by malware.

6.1.4 Network and Communication Artefacts

The `netscan` plugin recognised only loopback and VirtualBox host-only network interfaces, and no external IP connections or open ports. As networking was deliberately disabled within the virtual environment to prevent malware from communicating with the host machine, this lack of traffic was expected. Environment variable data showed the system was set up as a local session with no domain connection. These network artefacts represent an expected foundation for an isolated system configuration and show how Volatility records networks in a contained system.

This baseline is a suitable reference for identifying any command-and-control or remote access artefacts if networking was enabled in the infected analysis.

6.1.5 Credential and Security Artefacts

All security and credentials plugins (hashdump, lsadump, cachedump, privileges, getsids) executed successfully and Windows authentication information and privilege tokens were recovered. These represent artefacts in memory where credentials, cached logons, and LSA secrets can be parsed. Successfully extracting them means that Volatility can interpret the Windows security system under normal conditions as expected and demonstrate a basis in which corruption or inaccessibility within an infected image can be identified, a result expected of a kernel or registry modification by PlugX.

6.1.6 File System and Persistence Artefacts

Persistence and file system plugins filescan, mftscan, handles, modules, callbacks, and ssdt showed regular system files, open handles, and usual kernel drivers, and no unexpected hooks or injected modules were identified, as expected. Modified drivers, system callbacks, or SSDT entries are persistence mechanisms where artefacts might be identified. The results here act as a baseline reference for the infected image, where these areas can be analysed for missing or corrupted entries.

6.1.7 Baseline Artefact Reference Table

Category	Example Artefacts	Key Plugins	Comparison Relevance
<i>System Information</i>	OS build data, kernel address, registry hives	windows.info, windows.verinfo, windows.hivelist	Demonstrates normal kernel and registry structures as a reference for corruption
<i>Process Execution</i>	System, winlogon.exe, explorer.exe, smss.exe, svchost.exe, threads, loaded DLLs	pslist, psscan, dlllist, cmdline	Defines normal process hierarchy and execution for comparison to hidden or injected processes
<i>Network Configuration</i>	Local loopback, TCP/UDP endpoints (none external)	netscan, envvars	Reference for C2 or remote session artefacts
<i>Credentials & Security</i>	Password hashes, cached credentials, LSA secrets, SIDs, privilege tokens	hashdump, lsadump, cachedump, getsids, privileges	Shows security subsystem and credential stores; a baseline for highlighting inaccessible or modified areas
<i>File System & Persistence</i>	Handles, registry keys, MFT, kernel modules, SSDT, callbacks	filescan, mftscan, handles, modules, callbacks, ssdt	Comparison points for showing missing modules, hidden drivers, or persistence hooks

6.1.8 Interpretation Summary

The clean image represents a baseline reference map of volatile artefacts within a clean Windows 10 system. All plugins were executed successfully and produced expected results, assuring the integrity of the kernel and highlighting the accessibility of normal artefact locations. This foundation demonstrates what Volatility can recover and defines expected sources of evidence that can be later examined for absence or corruption in the infected PlugX image (Volatility Foundation, 2023; Case and Richard, 2017).

6.2 Infected Memory Image

6.2.1 Overview

The malware-infected Windows 10 memory image was analysed using the same Volatility 3 plugin choices as the clean image baseline. Despite the identical conditions the data was collected in and a verified image hash, the infected image failed to produce any artefact data. Volatility recognised the image and began running plugins, but all failed across each category, likely due to kernel layer and symbol table requirements not being met. The infected image, therefore, cannot be parsed into interpretable system structures. Despite this limitation, analysis can still take place to some degree.

6.2.2 Consistency of Failures

All executions were terminated with a similar framework error, typically:

*“Unable to validate the plugin requirements:
[‘plugins.<PluginName>.kernel.layer_name’,
‘plugins.<PluginName>.kernel.symbol_table_name’]”*

The error message appeared in every output file. Each plugin completed the file scanning and PDB lookup steps and then failed when attempting to map kernel layers. No partial artefacts or truncated data were produced by any plugin, demonstrating that the error was not unique to individual modules. The consistency of the error suggests it was caused by a single failure; the software was unable to get a valid kernel context that was required for the following interpretation.

6.2.3 Nature and Extent of Failures

The resulting failures during analysis of the infected image were all roughly the same and occurred at the same executing stage for each plugin. Each case saw Volatility begin the initial setup, detect the image type, scan the file header, and try to stack the kernel layer before terminating with the same errors. Not even partial results were generated, meaning no residual artefacts were recorded in the output files.

This suggests that Volatility was able to recognise the memory image but could not proceed past kernel mapping. The point of termination remained the same across all

plugins, including those for system information, process enumeration, and artefact extraction. The logs indicate that each plugin was able to reach the translation-layer validation step, but stopped when it was unable to find the required `kernel.layer_name` and `kernel.symbol_table_name` parameters.

The common behaviour shows the issue affected Volatility as a whole rather than individual plugins. As a result, the infected image produced no recoverable artefacts, making it mostly forensically useless with standard Volatility 3 execution.

6.2.4 Dataset Overview and Verification

Although no valid artefacts were produced, the resulting plugin outputs can still be a part of the overall analytical dataset. Plugin execution created text files that contain framework logs and error statements, and are organised into the same Info, Scan, and Artefacts folders used for the clean image. The maintained structure allowed direct comparison between the corresponding plugins and ensured each analysis stage was performed under the same conditions.

In total, thirty-six output files and three summary logs were reviewed to ensure integrity and verify that no corruption had occurred upon export. The consistently “unsatisfied requirement” error messages demonstrate Volatility executing correctly but failing equally during kernel initialisation. Preservation of produced outputs provides a record for auditing the infected image’s state before analysis and helps form an evidentiary basis for explanation in Section 6.3.

6.2.5 Interpretation Summary

In summary, the analysis of the infected image confirmed that, despite the same conditions as the clean image, Volatility 3 was unable to extract any useful data regarding artefacts or system structures. The consistency and completeness of the failures demonstrate a systemic issue and were what prevented parsing at the kernel-mapping stage. While this limited evidence collection and analysis, the outputs can still provide insight into how the malware affected the analysis. The next section, 6.3, will examine the causes for these failures in technical detail.

6.3 Analysis of Failure Causes

6.3.1 Overview

Consistent plugin failure suggests a systemic issue within the software’s ability to interpret memory structures. Each plugin was terminated with “unsatisfied requirement” errors related to the `kernel.layer_name` and `kernel.symbol_table_name` parameters. These are required for Volatility to locate kernel objects, resolve memory offset, and convert virtual addresses into structures. The corruption of these components prevented further analysis and is the most likely cause of the failure.

6.3.2 Volatility Dependencies and Symbol Table Requirements

Volatility 3 uses a layered memory model that requires correctly identifying the kernel layer and the correct symbol table that corresponds to it. A symbol table contains defined memory offsets and information on data types that is extracted from Windows debugging files (PDBs) and is used to map kernel objects, including processes, threads, and drivers (Volatility Foundation, 2023). When the stored reference cannot be validated, it causes plugins to be unable to interpret object hierarchies or memory regions.

The repeated “unsatisfied requirement” errors in the infected image outputs strongly suggest the layer relationship described here was broken early into parsing. Given the same PDB and plugin set functioned perfectly for the clean image, we can rule out tool misconfiguration as a cause. The plugin failure must therefore originate from corrupted or altered kernel symbols within the memory image, the result of which is Volatility's inability to build a valid translation layer.

6.3.3 Kernel Disruption and Address Translation Failures

The errors observed in outputs are in line with situations where the kernel's internal address space or page tables have been modified or corrupted. PlugX malware works to perform in-memory process injection and code hollowing and has the capability to overwrite or unlink kernel structures like object directories and page tables (Unit42, 2021; Volatility Labs, 2015). Manipulations such as these could cause inconsistencies between what Volatility would expect to find in memory and what actually exists there.

As Volatility attempts to validate kernel offsets during layer processing, the changed data stops the software from resolving the addresses, resulting in termination. This corruption also removes fallback strategies (like scanning for signature-based artefacts) as an option, as the translation between virtual and physical addresses is lost or unreadable.

6.3.4 Tool Limitations and Reasoning for Error Propagation

While corruption of kernel structure is very likely the root of the issue, the constituent failures of all plugins show a limitation in Volatility's dependency handling. It runs each plugin using shared base layers, which likely causes every subsequent plugin to inherit the same error when a single dependency, such as the kernel layer in this case, fails to initialise (Case and Richard, 2017). This is a strong indication of why each plugin was unable to produce any results.

The results highlight both the effect of kernel disruption and the rigidity of Volatility layer validation. This design likely prevents the processing of incomplete or non-standard kernel mapping, which is reasonable in typical investigations, but results in constrained analysis when advanced obfuscation techniques (like the ones used by PlugX) are met.

6.3.5 Summary

In summary, the consistency of plugin errors from the infected image is plausibly explained by Volatility being unable to initialise the kernel layer or the symbol table dependencies required for memory translation, as the errors would suggest. The failure pattern supports the assumption that kernel structures or memory mappings were altered or corrupted and therefore prevented address resolution. While logical, these explanations cannot be verified absolutely with the dataset collected; they are, however, supported by known behaviours of in-memory malware such as PlugX. Published analyses describe how PlugX modifies kernel structures, page tables, and non-paged pool memory in an effort to conceal itself (Trend Micro, 2020; Unit 42, 2021; Volatility Labs, 2015). The commonality of these analyses and our observed Volatility failures strongly suggests a relationship between the two. It is important to note, however, that this is an informed inference and does not represent confirmed evidence-based findings. Section 6.4 explores this by considering PlugX's Behaviour and its implications in greater detail.

6.4 Forensic Implications of PlugX Behaviour

6.4.1 Technical Functionality

PlugX operates entirely within volatile memory to avoid detection and forensic evidence recovery. When executed, the loader decrypts the payload and uses DLL sideloading to load a library, pretending to be a legitimate signed executable, usually `explorer.exe` or `svchost.exe` (Cybereason, 2022). When loaded, it uses process injection and code hollowing techniques to overwrite the legitimate process's memory with its own predetermined instructions while maintaining the original process identifiers (Trend Micro, 2020).

PlugX then attempts to create encrypted C2 communications and injects additional modules that expand its functionality. These additional modules change depending on the version of PlugX but are often keylogging, screen capture, and file exfiltration (Unit 42, 2021). During this process, the malware allocates and changes non-paged pool memory and heap regions. These manipulations are intended to remove traces of the injected threads and allocated regions from the system's enumeration tables (Volatility Labs, 2015).

Forensically, this means the malware's artefacts exist only as encrypted or transient memory data. When forensic tools attempt to interpret these areas, they will find incomplete or inconsistent structures, resulting in read errors or a lack of output. The malware, therefore, conceals its executions and also prevents forensic parsing after an incident (Volatility Foundation, 2023).

6.4.2 Impact on Volatility and Forensic Results

The behaviours outlined in Section 6.3 are related to the expected forensic results of PlugX's in-memory modifications. Alteration or unlinking kernel structures causes the malware to create conditions where memory analysis tools cannot resolve object offset or validate symbol tables, resulting in the observed outputs.

The lack of output data itself presents two significant forensic implications. Firstly, the absence of readable data itself is an indicator of malware infection, revealing that malicious code has interfered at a significant enough level capable of corrupting system artefacts, and demonstrating the complexity of the malware. Secondly, it highlights a limitation of automatic memory-forensic software that requires static kernel analysis. In practice, if this kind of failure occurs, investigators should check their results using alternative forensic tools or kernel profiles and, if possible, capture memory images at different times during execution before malware has fully loaded itself into memory.

7. Discussion

The investigation successfully demonstrated how volatile memory analysis can reveal critical artefacts from a live system, even in the absence of disk-based data. Using Volatility 3, the project extracted details on running processes, network connections, and command histories, showing how analysts can reconstruct system activity solely from RAM.

The automated scripts (`vol3_info.sh`, `run_vol3_scan.sh`, and `vol3_compare.sh`) streamlined the forensic workflow by standardising data extraction and reducing manual effort. This scripting approach reflects real-world practices in SOCs and CSIRTs, where repeatable automation ensures consistency across multiple memory images. While the baseline image produced consistent results, the malware-injected image caused plugin validation errors, suggesting symbol table incompatibility or corruption of memory structures. This behaviour illustrates how certain malicious activities or altered system states can hinder traditional forensic tooling—an important lesson in tool limitation awareness.

The optional comparison script (`vol3_compare.sh`) was conceptualised to extend the project toward differential analysis between pre- and post-compromise states. Although not executed within the current scope, it demonstrates how future work could leverage automated diffs to identify newly spawned processes, injected DLLs, or changed network artefacts, supporting faster triage and incident correlation.

7.1 Comparison to Literature and Case Studies

According to the Threat Encyclopedia published by TrendMicro, PlugX has been reported to delete multiple non-malicious files before creating replacement files with similar or identical names with the intention of disguising malicious programs as expected Windows files. The malware is also known to create backdoors and automation of self-execution by editing the registry file `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services` to include subfiles that would result in each of the malicious programs being run on system startup (TrendMicro, 2016). These alterations to the registry were not able to be recorded by Volatility during this project due to the continual error, but the reporting of created programs and file paths for each respective program, as well as the recording or alterations to the registry, indicates that there are other digital forensic tools or methods capable of analysing the processes of PlugX.

Real-world forensic reports have also noted that PlugX will scan the current processes of the machine, and if any virtual machine-related processes are found, it will not continue with the expected infection procedure (Tom Lancaster, Esmid Idrizovic, 2017).

Checking a VM running in the background before initiating malicious activity is a method of preventing the malware from being analysed in an isolated environment that, in the case of a successful analysis, would result in defensive measures being put in place against PlugX. This anti-forensic method may have been the reason for all of the output resulting in the same two errors, but the found real-world report does not specify the actions that PlugX would take after detecting the presence of a VM; it cannot be confirmed as the reason for the failures seen in this project. Two processes running in the background that were recorded from the clean memory image were VBoxService.exe and VBoxTray.exe. These process listings can be seen in Figure 1. It is not known how to prevent the malware from detecting the background VM processes, but the potential reason for consistent errors in all Volatility output when analysing the infected memory capture is a good indication of anti-forensics and is a point of further investigation.

```
1288    572 VBoxService.exe 0xb78aa587a080 12 - 0 False 2025-10-22 23:04:56.000000 UTC N/A Disabled
4816   3216 VBoxTray.exe 0xb78aa6122080 15 - 1 False 2025-10-22 23:05:54.000000 UTC N/A Disabled
```

Figure 1 – Table listings of processes run by VirtualBox printed by the windows.pslist Volatility 3 plugin. Table column order follows left to right: PID, PPID, Image File Name, Offset, Threads, Handles, Session ID, Wow64, Create Time, Exit Time, and File Output.

Unfortunately, this project was unable to complete a full analysis on PlugX due to the continuous errors produced when analysing the infected memory capture. However, the points of failure also lead to potential insight into how the malware is programmed to detect and prevent dynamic or other forensic analysis of it, thus preventing countermeasures from being implemented into systems. Reconfiguration of the Windows 10 VM, before continual investigations into how PlugX functions, will be required and will provide an opportunity for greater understanding of the malware.

7.2 Recommendations and Future Improvements

Several limitations impacted the project outcomes. First, the infected memory image failed plugin validation, preventing full comparative analysis. This highlights the dependency of forensic tools on intact kernel structures and symbol tables, which advanced malware can compromise. Second, the project only analysed a single malware sample in a controlled VM, limiting the generalisability of results to real-world environments with multiple, interacting threats.

Future work could address these limitations by testing multiple memory images at different stages of infection, integrating alternative forensic tools to bypass kernel corruption, and expanding the automation scripts to handle diverse memory formats. Enhancements could also include implementing custom Volatility plugins for PlugX-specific artefacts, integrating network telemetry for correlated analysis, and using

snapshot comparisons for incremental memory analysis. These improvements would create a more robust framework for both educational and operational memory-forensics investigations.

7.3 Reflective Analysis

The project highlighted the critical importance of tool selection and validation in digital forensics. While Volatility 3 provided a powerful platform for analysing memory artefacts, the errors encountered with the infected image underscored how fragile such analyses can be when facing corrupted or obfuscated kernel structures. This reinforces the principle that forensic tools must be verified against known baselines before deployment, and analysts should understand the assumptions and limitations underlying each plugin. Effective validation ensures that extracted evidence remains reliable and admissible in investigative contexts.

Another key learning was the limitation of automated analysis. The developed Bash scripts successfully standardised and accelerated the forensic process, yet automation alone cannot guarantee complete or accurate insight. Some plugin outputs required contextual interpretation that automation could not provide. This experience demonstrated that while scripts can enhance efficiency, human expertise remains essential to interpret anomalies, recognise tool failures, and correlate artefacts across sources.

Finally, the project reaffirmed the need for layered evidence collection strategies. Relying solely on memory analysis, though insightful, offers only a partial view of system activity. Integrating volatile data with disk forensics, network telemetry, and log analysis would provide a more holistic understanding of compromise events. This layered approach aligns with professional Digital Forensics and Incident Response practices, ensuring that evidence gaps caused by memory corruption or tool incompatibility can be bridged using complementary data sources. Overall, the reflective process reinforced that effective memory forensics depends not only on technical execution but also on investigative adaptability, critical reasoning, and procedural rigour.

8. Conclusion

This project successfully achieved its primary objective of exploring the value of memory forensics in incident response and malware analysis. By acquiring and examining RAM images, the study demonstrated how volatile artefacts can expose system activities, reveal running processes, and highlight possible signs of compromise. Using Volatility 3 and scripted automation, the project replicated a miniature version of enterprise forensic workflows, reinforcing concepts such as evidence integrity, repeatability, and structured analysis. The results also emphasised common challenges, such as plugin dependency issues and symbol table validation, that practitioners must handle during real-world investigations.

While limited in scope, the project lays the groundwork for more advanced forensic studies involving automated artefact comparison and malware behaviour tracing. Future iterations could integrate custom Volatility plugins, enhanced automation for cross-image analysis, and integration with log or network telemetry for richer context.

In conclusion, this exercise highlighted how memory forensics, even when performed in a controlled academic setting, cultivates essential analytical and technical skills directly transferable to professional DFIR operations.

9. References

- Ashley P. 2021, *Volatility 3 CheatSheet*, viewed 13 October 2025, <https://blog.onfvp.com/post/volatility-cheatsheet/>.
- Oracle Corporation n.d., *VBoxManage debugvm - VirtualBox user manual*, Oracle, viewed 13 October 2025, <https://docs.oracle.com/en/virtualization/virtualbox/6.0/user/vboxmanage-debugvm.html>
- Volatility Foundation 2023, *Volatility 3 Framework Documentation*, Volatility Foundation, viewed 13 October 2025, <https://volatility3.readthedocs.io/>
- ytisf n.d., *theZoo - A Live Malware Repository*, GitHub, viewed 13 October 2025, <https://github.com/ytisf/theZoo>
- Cybereason 2022, *Threat Analysis Report: PlugX RAT Loader Evolution*, Cybereason Global SOC Team, 8 September 2022, viewed 13 October 2025, <https://www.cybereason.com/blog/threat-analysis-report-plugx-rat-loader-evolution>
- Malpedia (Fraunhofer FKIE) 2022, *PlugX (Malware Family)*, Malpedia, viewed 13 October 2025, <https://malpedia.caad.fkie.fraunhofer.de/details/win.plugx>
- REMnux n.d., *REMnux: A Linux toolkit for malware analysis*, REMnux, viewed 13 October 2025, <https://remnux.org/>
- Zeltser L. n.d., *5 Steps to Building a Malware Analysis Toolkit Using Free Tools*, zeltser.com, viewed 13 October 2025, <https://zeltser.com/build-malware-analysis-toolkit/>
- Ligh, M.H., Case, A., Levy, J. & Walters, A. 2014, *The Art of Memory Forensics: Detecting Malware and Threats in Windows, Linux, and Mac Memory*, Wiley, Indianapolis
- NIST (National Institute of Standards and Technology) 2023, *Guide to Integrating Forensic Techniques into Incident Response (SP 800-86 Rev. 1 Draft)*, US Department of Commerce, viewed 15 October 2025, <https://csrc.nist.gov/publications/>
- Case, A. & Richard, G.G. 2017, *Digital Forensics and Incident Response*, Syngress, Oxford
- Volatility Labs 2015, *PlugX: Memory Forensics Lifecycle with Volatility*, viewed 15 October 2025, <https://volatilityfoundation.org/plugx-memory-forensics-lifecycle-with-volatility/>
- Trend Micro 2020, *PlugX: A Stealthy and Persistent Remote Access Trojan*, Trend Micro Research, viewed 3 Nov 2025, <https://www.trendmicro.com/>
- Unit 42 (Palo Alto Networks) 2021, *PlugX Malware Evolution: New Variants and Techniques*, viewed 4 Nov 2025, <https://unit42.paloaltonetworks.com/>.
- Tom, L. & Esmid, I. 2017, *Paranoid PlugX*, viewed 7 November 2025, <https://unit42.paloaltonetworks.com/unit42-paranoid-plugx/>.

TrendMicro 2016, *PLUGX*, viewed 7 November 2025,
<https://www.trendmicro.com/vinfo/us/threat-encyclopedia/malware/plugx>.

10. Appendix

Appendix A - Hash Verification Logs

Time of Hash	Cleanmemdump.mem
11:12 am 23/10/2025	9bd2619410dd8438094bcb03c0125790ac9e45414c6b5e2bdce6121d64371d78 ./Desktop/Cleanmemdump.mem
5:20 pm 24/10/2025	9bd2619410dd8438094bcb03c0125790ac9e45414c6b5e2bdce6121d64371d78 ../volatility3/Cleanmemdump.mem
12:01 am 30/10/2025	9bd2619410dd8438094bcb03c0125790ac9e45414c6b5e2bdce6121d64371d78 /home/ubuntu_user/volatility3/Cleanmemdump.mem
2:05 pm 30/10/2025	9bd2619410dd8438094bcb03c0125790ac9e45414c6b5e2bdce6121d64371d78 /home/ubuntu_user/volatility3/Cleanmemdump.mem

Time of Hash	PlugXInfectedmemdump.mem
11:12 am 23/10/2025	f138d906d61d5da512ef6adbc054fd08cdca1dd05132fc0dd2106f09ba1564ba ./Desktop/PlugXInfectedmemdump.mem
5:20 pm 24/10/2025	f138d906d61d5da512ef6adbc054fd08cdca1dd05132fc0dd2106f09ba1564ba ../volatility3/PlugXInfectedmemdump.mem
12:01 am 30/10/2025	f138d906d61d5da512ef6adbc054fd08cdca1dd05132fc0dd2106f09ba1564ba /home/ubuntu_user/volatility3/PlugXInfectedmemdump.mem
2:05 pm 30/10/2025	f138d906d61d5da512ef6adbc054fd08cdca1dd05132fc0dd2106f09ba1564ba /home/ubuntu_user/volatility3/PlugXInfectedmemdump.mem

Appendix B – Software and Forensic Tools Used

VirtualBox – a virtual machine monitor for x86 and ARM virtualisation that was developed by the company Oracle. During this project, VirtualBox was to host the virtual machines as it provided a sufficient level of isolation from the host computer that ensured no live systems would be unintentionally compromised when executing the selected malware. VirtualBox can be downloaded from <https://www.virtualbox.org/wiki/Downloads>

Windows media creation tool – this tool allowed the download of the Windows 10 ISO that was later set up and run in VirtualBox to execute the PlugX malware. This tool is owned and maintained by Microsoft and can be downloaded from <https://www.microsoft.com/en-au/software-download/windows10>

Ubuntu – run with VirtualBox and was used to run Volatility 3 to analyse the captured memory files. Ubuntu can be downloaded from <https://ubuntu.com/download/desktop>

Volatility 3 – command line run memory artefact extraction tool. This tool was run in Ubuntu and used to analyse captured memory files of the Windows 10 VM before and after infecting the VM with PlugX. Volatility can be downloaded from

<https://github.com/volatilityfoundation/volatility3.git>

FTKImager – computer memory forensic software owned by Exterro. FTKImager is able to capture the memory of the system running the software, as well as analyse a given memory capture. FTKImager can be downloaded from

<https://www.exterro.com/ftk-product-downloads/ftk-imager-4-7-3-81>